order of several times $N \log_2 N$ operations. The algorithm inventor tries to reduce the constant in front of this estimate to as small a value as possible. Two of the best algorithms are *Quicksort* (§8.2), invented by the inimitable C.A.R. Hoare, and *Heapsort* (§8.3), invented by J.W.J. Williams.

For large N (say > 1000), Quicksort is faster, on most machines, by a factor of 1.5 or 2; it requires a bit of extra memory, however, and is a moderately complicated program. Heapsort is a true "sort in place," and is somewhat more compact to program and therefore a bit easier to modify for special purposes. On balance, we recommend Quicksort because of its speed, but we implement both routines.

For small N one does better to use an algorithm whose operation count goes as a higher, i.e., poorer, power of N, if the constant in front is small enough. For $N < 20$, roughly, the method of *straight insertion* (§8.1) is concise and fast enough. We include it with some trepidation: It is an N^2 algorithm, whose potential for misuse (by using it for too large an N) is great. The resultant waste of computer resource can be so awesome that we were tempted not to include any N^2 routine at all. We *will* draw the line, however, at the inefficient N^2 algorithm, beloved of elementary computer science texts, called *bubble sort*. If you know what bubble sort is, wipe it from your mind; if you don't know, make a point of never finding out!

For $N < 50$, roughly, *Shell's method* (§8.1), only slightly more complicated to program than straight insertion, is competitive with the more complicated Quicksort on many machines. This method goes as $N^{3/2}$ in the worst case, but is usually faster.

See Refs. [1,2] for further information on the subject of sorting, and for detailed references to the literature.

CITED REFERENCES AND FURTHER READING:

Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley).[1]

Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), Chapters 8–13.[2]

8.1 Straight Insertion and Shell's Method

Straight insertion is an N^2 routine and should be used only for small N, say < 20.

The technique is exactly the one used by experienced card players to sort their cards: Pick out the second card and put it in order with respect to the first; then pick out the third card and insert it into the sequence among the first two; and so on until the last card has been picked out and inserted.

sort.h

```
template<class T>
void piksrt(NRvector<T> &arr)
```

Sort an array `arr[0..n-1]` into ascending numerical order, by straight insertion. `arr` is replaced on output by its sorted rearrangement.

```
{
    Int i,j,n=arr.size();
    T a;
    for (j=1;j<n;j++) {                    Pick out each element in turn.
        a=arr[j];
```

```
        i=j;
        while (i > 0 && arr[i-1] > a) {     Look for the place to insert it.
            arr[i]=arr[i-1];
            i--;
        }
        arr[i]=a;                           Insert it.
    }
}
```

Notice the use of a template in order to make the routine general for any type of `NRvector`, including both `VecInt` and `VecDoub`. The only thing required of the elements of type `T` in the vector is that they have an assignment operator and a > relation. (We will generally assume that the relations <, >, and == all exist.) If you try to sort a vector of elements without these properties, the compiler will complain, so you can't go wrong.

It is a matter of taste whether to template on the element type, as above, or on the vector itself, as

```
template<class T>
void piksrt(T &arr)
```

This would seem more general, since it will work for any type `T` that has a subscript operator `[]`, not just `NRvectors`. However, it also requires that `T` have some method for getting at the type of its elements, necessary for the declaration of the variable `a`. If `T` follows the conventions of STL containers, then that declaration can be written

```
T::value_type a;
```

but if it doesn't, then you can find yourself lost at C.

What if you also want to rearrange an array `brr` at the same time as you sort `arr`? Simply move an element of `brr` whenever you move an element of `arr`:

sort.h

```
template<class T, class U>
void piksr2(NRvector<T> &arr, NRvector<U> &brr)
```

Sort an array `arr[0..n-1]` into ascending numerical order, by straight insertion, while making the corresponding rearrangement of the array `brr[0..n-1]`.

```
{
    Int i,j,n=arr.size();
    T a;
    U b;
    for (j=1;j<n;j++) {                     Pick out each element in turn.
        a=arr[j];
        b=brr[j];
        i=j;
        while (i > 0 && arr[i-1] > a) {     Look for the place to insert it.
            arr[i]=arr[i-1];
            brr[i]=brr[i-1];
            i--;
        }
        arr[i]=a;                           Insert it.
        brr[i]=b;
    }
}
```

Note that the types of `arr` and `brr` are separately templated, so they don't have to be the same.

Don't generalize this technique to the rearrangement of a larger number of arrays by sorting on one of them. Instead see §8.4.

8.1.1 Shell's Method

This is actually a variant on straight insertion, but a very powerful variant indeed. The rough idea, e.g., for the case of sorting 16 numbers $n_0 \ldots n_{15}$, is this: First sort, by straight insertion, each of the 8 groups of 2 (n_0, n_8), (n_1, n_9), $\ldots$, (n_7, n_{15}). Next, sort each of the 4 groups of 4 (n_0, n_4, n_8, n_{12}), $\ldots$, $(n_3, n_7, n_{11}, n_{15})$. Next sort the 2 groups of 8 records, beginning with $(n_0, n_2, n_4, n_6, n_8, n_{10}, n_{12}, n_{14})$. Finally, sort the whole list of 16 numbers.

Of course, only the *last* sort is *necessary* for putting the numbers into order. So what is the purpose of the previous partial sorts? The answer is that the previous sorts allow numbers efficiently to filter up or down to positions close to their final resting places. Therefore, the straight insertion passes on the final sort rarely have to go past more than a "few" elements before finding the right place. (Think of sorting a hand of cards that are already almost in order.)

The spacings between the numbers sorted on each pass through the data (8,4,2,1 in the above example) are called the *increments*, and a Shell sort is sometimes called a *diminishing increment sort*. There has been a lot of research into how to choose a good set of increments, but the optimum choice is not known. The set $\ldots, 8, 4, 2, 1$ is in fact not a good choice, especially for N a power of 2. A much better choice is the sequence

$$(3^k - 1)/2, \ldots, 40, 13, 4, 1 \tag{8.1.1}$$

which can be generated by the recurrence

$$i_0 = 1, \qquad i_{k+1} = 3i_k + 1, \quad k = 0, 1, \ldots \tag{8.1.2}$$

It can be shown (see [1]) that for this sequence of increments the number of operations required in all is of order $N^{3/2}$ for the worst possible ordering of the original data. For "randomly" ordered data, the operations count goes approximately as $N^{1.25}$, at least for $N < 60000$. For $N > 50$, however, Quicksort is generally faster.

sort.h

```
template<class T>
void shell(NRvector<T> &a, Int m=-1)
```

Sort an array `a[0..n-1]` into ascending numerical order by Shell's method (diminishing increment sort). `a` is replaced on output by its sorted rearrangement. Normally, the optional argument `m` should be omitted, but if it is set to a positive value, then only the first `m` elements of `a` are sorted.

```
{
    Int i,j,inc,n=a.size();
    T v;
    if (m>0) n = MIN(m,n);                  Use optional argument.
    inc=1;                                  Determine the starting increment.
    do {
        inc *= 3;
        inc++;
    } while (inc <= n);
    do {                                    Loop over the partial sorts.
        inc /= 3;
        for (i=inc;i<n;i++) {               Outer loop of straight insertion.
            v=a[i];
            j=i;
            while (a[j-inc] > v) {          Inner loop of straight insertion.
                a[j]=a[j-inc];
                j -= inc;
                if (j < inc) break;
            }
```